Do Self-Improved Repair Policies Transfer to Real Bugs? A Preregistered External-Benchmark Test of a Level-4 Bounded Self-Improving System

Junai P. Felix*
Independent Researcher, Taipei, Taiwan

October 2026
Version 1.1

Abstract

A system that improves itself inside a synthetic training domain may only be getting better at that domain. We test whether the gains of FELIX-C3 carry over to real bugs. FELIX-C3 is a self-modifying program-repair system that meets Liu et al.'s (2026) Level-4 definition of bounded recursive self-improvement on generated tasks. We took the 72 final policies of its published confirmatory missions (three conditions $\times$ 24 seeds) and verified each against the published audit manifest. We then ran them, unmodified, on 164 valid bugs from two established external benchmarks: QuixBugs and HumanEvalFix, both in Python. The only new code is a sandboxed executor and a port of the frozen search; the port reproduced the original search exactly in 1,200 of 1,200 synthetic searches. The protocol, primary endpoint, decision rule and development evidence were locked and hashed before any confirmatory policy was evaluated.

Under the identical search budget, the self-improved policies solved 25.8 bugs against 21.0 for the system's own starting policy: +4.85 bugs, 95% CI [3.80, 5.84], with 23 of 24 seeds improved (exact one-sided Wilcoxon $p = 3.0 \times 10^{-7}$). That is a relative gain of 23%, against 47% on the synthetic domain. The gain:

- held in 69 of 72 missions;
- persisted at four times the training budget (+2.17);
- disappeared when the learned parameters were permuted (5.97 bugs below the trained policy);
- came entirely from HumanEvalFix (+5.08), with none on QuixBugs (-0.23).

The three conditions did not differ (C6 $-$ C6f +0.26, $p = 0.68$; C6 $-$ C6r -0.67, $p = 0.35$). A fresh-process replay reproduced every audited unit exactly. A registered secondary on 72 independent end states from a later study of the same system replicated both findings: +5.0 bugs over the starting policy, and no difference between an evolved and a default improvement mechanism.

The self-improvement is therefore not confined to the synthetic domain. What transfers, however, is the product of the bounded improvement loop, not a benefit of the system rewriting its own improvement mechanism. The system also remains a one-step repairer that can reach at most 55 of the 164 bugs.

Keywords: recursive self-improvement · external validity · transfer · automated program repair · QuixBugs · HumanEvalFix · preregistration

*Correspondence: junai.felixx@gmail.com.

Contents

1 Introduction **3**

2 Background and Related Work **4**

3 System Under Test and the Port **5**
- 3.1 Disclosure boundary 5
- 3.2 What is frozen 5
- 3.3 What is new: execution and the port 5

4 Benchmarks and Tasks **7**

5 Design and Preregistration **8**
- 5.1 Population, baseline and control 8
- 5.2 Units, streams and blocks 8
- 5.3 Primary endpoint and decision rule 8
- 5.4 Secondary analyses 8
- 5.5 What was known before the lock 9
- 5.6 Timeline 9

6 Results I: Gate 0 and Integrity **10**

7 Results II: Transfer **11**

8 Results III: Where the Gain Is **14**

9 Discussion **16**

10 Limitations **17**

11 Reproducibility, Integrity and Use of AI Assistance **18**

12 Conclusion **19**

A Per-Seed Results **22**

B Hashes **23**

1 Introduction

Self-improving systems are usually evaluated where they improve. A system that rewrites its own policy, generator or improvement procedure on a stream of generated tasks, and gets better at those tasks, has shown that it can climb its own training signal. It has not shown that the climb was toward anything outside it. This gap matters most where self-improvement is claimed, because a system tuned to its own task generator can look like it is improving without having become more capable in the sense anyone cares about.

The Stage 3 study of this programme [5] preregistered a test of FELIX-C3 against Liu et al.'s Level 4, *bounded recursive self-improvement* [10]. The system rewrote its own improvement mechanism, and over 32 autonomous rounds it raised its held-out solve rate on a synthetic program-repair domain from 27.4% to 40.1%. Every definitional criterion passed. The causal criteria did not: rewriting the mechanism was not shown to cause the progress. All of those measurements were taken inside the synthetic domain the system trained on.

This paper asks the external-validity question directly. **Do the repair policies the system produced by self-improvement repair more real bugs, drawn from established external benchmarks, than the system's own starting policy, under the identical frozen search?** A yes means part of the self-improvement is repair competence and not just fit to the task generator. A no means the gain stayed in the sandbox. We registered the test so that either answer would be reported.

Contributions.

1. A preregistered external-benchmark test of a published self-improving system. It uses the system's own 72 confirmatory end states, verified by hash against the published audit manifest, together with a fixed set of 164 real bugs from QuixBugs [9] and HumanEvalFix [13].
2. A port of the frozen search to real code that is shown to be *exact*. On synthetic tasks it reproduced 1,200 of 1,200 searches and all 2,121 diagnostic computations. The task-domain code the policies were trained on and the frozen code agree on all 6,514 real-code mutation pools.
3. A positive primary result: +4.85 bugs (+23%), $p = 3.0 \times 10^{-7}$. A negative control shows the gain comes from what was learned. The result also has two clear limits: no transfer to QuixBugs, and no difference between conditions. Both the transfer and the null between conditions replicate on 72 independent missions.
4. A complete integrity record: Gate 0 checks, a fresh-process replay, the development evidence disclosed before the lock, and an accurate statement of AI assistance.

2 Background and Related Work

The system and its Level-4 result. FELIX-C3 is the third stage of a programme that tests self-improving runtime modules against published capability definitions [5]. A repair *task policy* decides in which order a fixed repair search tries candidate edits. Above the policy are a generator that shapes which policy revisions can be expressed, and an explicit, versioned improvement mechanism (Imp). Imp contains the procedure that edits Imp itself. Three conditions were run on 24 seeds each:

- C6: Imp rewritable on evidence;
- C6f: Imp frozen;
- C6r: Imp rewritten, with adoption by coin flip.

All three improved strongly on synthetic held-out tasks, and the definitional Level-4 criteria passed. C6 did not significantly beat either control (+0.35 pp, $p = 0.23$; +0.72 pp, $p = 0.079$). Three further preregistered studies have not shown the recursion to be productive either: a transplant test of evolved mechanisms, and two redesigns of how rewrites are validated. A third redesign is in progress.

External validity. The synthetic domain has 16 parametric families of short functions with one to three injected faults. Four families are held out from training, so "held-out" already meant more than new instances. But every family comes from the same generator and the same injected-fault model. Real bugs differ in program size, control flow (loops, recursion, strings, nested data), the vocabulary of operations, and how faults are distributed. A policy that learned regularities of the generator could fail on them entirely.

Benchmarks. QuixBugs [9] has 40 classic algorithms, each with a one-line bug, in Python and Java. HumanEvalFix, the repair task of HumanEvalPack [13], adds a manually written bug to each of the 164 HumanEval [1] reference solutions. Each bug is labelled with one of six types: value misuse, operator misuse, variable misuse, function misuse, missing logic and excess logic. Both are widely used to evaluate repair systems, including LLM-based ones. Neither was used at any point in developing FELIX-C3.

Search-based and learned repair. Generate-and-validate repair searches a space of candidate edits and checks each against tests. It ranges from genetic improvement [7] to mutation-based fixing [2]; see the reviews by Le Goues et al. and Monperrus [8, 12]. The competent-programmer hypothesis [3, 6] holds that many real faults are small deviations from a correct program, which is why small-edit searches find real fixes at all. Learning which candidates to try first is an established idea; Prophet learns a ranking of candidate patches from human fixes [11]. In our setting the ranking policy was never trained on real fixes. It was produced by a self-improvement loop on synthetic tasks, and the question is whether that ranking happens to be useful on real bugs.

Preregistration. As in Stage 3 we follow preregistration practice [14, 15]. We locked the question, the population, the endpoint, the decision rule and the analysis plan and hashed them with the code before any confirmatory evaluation. We also disclose the exploratory development work that shaped the plan.

3 System Under Test and the Port

3.1 Disclosure boundary

As in Stage 3, we describe FELIX-C3 at the level of roles and interfaces. We withhold the representation of the task policy, the form of its features and vocabulary, and how policy revisions are constructed. What this test needs is reported in full: which policies were run, how they were run, on what, with which budget, and every result.

3.2 What is frozen

A *repair attempt* takes a faulty function, its visible examples and a task policy. It runs a first-improvement hill climb over single-node syntactic edits of the function. The edits change:

- an integer constant by ± 1 or ± 2;
- an arithmetic operator within $\{+, -, \times, //, \%\}$;
- a comparison operator;
- a Boolean connective;
- a call among a small set of built-ins;
- the branches of a conditional expression;
- the removal of a unary minus;
- a variable among the function's parameters and comprehension variables.

At each step the task policy orders the candidate edits using descriptors of the edit and diagnostics of how the current program fails. The search evaluates candidates in that order until one passes more visible examples than the current program, accepts that edit, and continues from the edited program. It stops when all visible examples pass, when no edit improves, or when the *evaluation budget* of 12 candidate evaluations per task is spent. A task is *solved* if the final program passes every visible and every held-back example.

The edit set, the descriptors, the diagnostics, the search and the budget are part of FELIX-C3's fixed task domain. None of them was changed for this study. The policies themselves are the end states of the Stage 3 missions, used exactly as saved.

3.3 What is new: execution and the port

The synthetic domain runs candidates in-process under a restricted interpreter. Real benchmark code needs the full language, so candidates now run in a long-lived sandboxed worker process with:

- a per-example time limit of 1.0 s, about 30 times the slowest reference program; at the first timeout the candidate stops and its remaining examples count as failed;
- a 2 GB address-space limit;
- a recursion limit of 3,000;
- automatic restart of the worker if it stalls.

The failure diagnostics that the policy reads were re-implemented on the worker's outputs, tag for tag. Every candidate verdict is stored, keyed by task and source hash, together with the wall time of the run that produced it.

Two Gate 0 checks (Section 6) establish that the port changes nothing but the executor:

- **Exactness.** On 300 synthetic tasks under four policies, the ported search returns the same solved flag, the same number of evaluations and the same final program as the original search in every case. The re-implemented diagnostics equal the originals at every checked program.
- **Host equivalence.** The Stage 3 policies were trained against the Stage 3 task-domain code. The study uses the later version, which is faster but meant to produce identical output. The two give identical candidate pools and features on every real buggy function and on every one of its one-step edits.

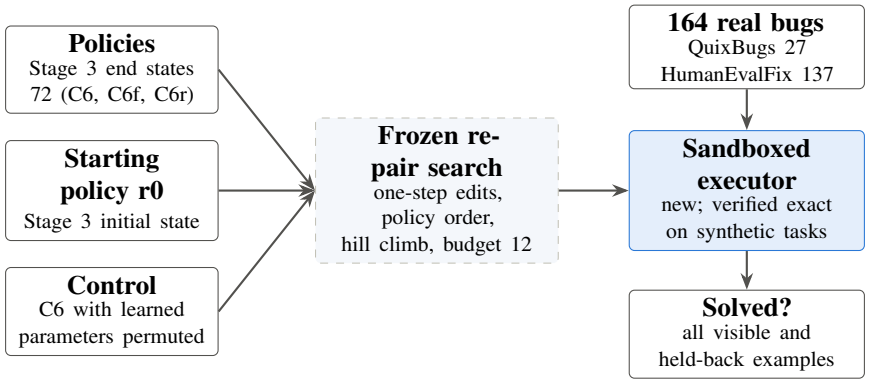


Figure 1: The test. Policies saved by the Stage 3 missions and the system's own starting policy run through the unchanged repair search on real bugs. Only the executor is new (blue); the search is part of the fixed task domain (dashed).

4 Benchmarks and Tasks

Sources. Both benchmarks are MIT-licensed, and both are pinned:

- **QuixBugs (Python).** Git commit `4257f44b`. Its JSON test cases are used.
- **HumanEvalFix (Python split of HumanEvalPack).** The parquet file is pinned by SHA-256 (Appendix B). Its examples are parsed from the `check` assertions of the form `candidate(args) == literal`, `candidate(args)` and `not candidate(args)`, with literal arguments.

For each task the faulty function is separated from any helper code. Only the faulty function is edited; helpers, imports and module code are kept as context.

Validity. A bug is *valid* if it meets three conditions:

1. it has at least four parsable examples;
2. the reference fix passes all of them;
3. the faulty program fails at least one.

Table 1 gives the counts. The valid set and a hash of its full content were frozen. Validation was repeated in three fresh processes with identical results.

Table 1: Benchmark composition. Examples are split in their original order: the first $\lceil n/2 \rceil$ are visible to the search and the rest are held back.

	Loaded	Too few examples	Reference fails	**Valid**	Median examples
QuixBugs	31	1	3	**27**	7 (4–14)
HumanEvalFix	164	27	0	**137**	7 (4–26)
Total	195	28	3	**164**	

Split. The visible examples are the first $\lceil n/2 \rceil$ in original order, and the rest are held back. This mirrors the synthetic domain, where a repair must generalise from visible to held-back examples. In 15 of the 164 bugs, the faulty program already passes every visible example. These bugs cannot be solved by any policy, because the search sees nothing to fix. They are kept in the denominator.

Reachability (Phase 0, no policies). Before any policy was evaluated, we counted the bugs that have a one-step fix within the frozen edit set, with no budget limit. There are 55 such bugs: 4 of 27 in QuixBugs and 51 of 137 in HumanEvalFix. This roughly caps what any policy can achieve, and it makes the absolute numbers below small by design. FELIX-C3 is a vehicle for testing self-improvement, not a competitive repair system.

5 Design and Preregistration

5.1 Population, baseline and control

Policies under test. The policies under test are the 72 round-32 policies of the Stage 3 confirmatory missions: arms C6, C6f and C6r, seeds 7101–7124. Before any use, every state file is checked by SHA-256 against the audit manifest published with Stage 3. Gate 0 reads only their hashes. No confirmatory state was parsed or evaluated on real bugs before the lock.

Baseline r0. The baseline is the Stage 3 initial state: an empty policy with the designer vocabulary. It is the same for every mission, and its candidate order is set only by the tie-break stream. It is therefore the system's own starting point, and the natural reference for "did self-improvement help?".

Control. For each C6 mission, the learned parameters are permuted among the same features, using a fixed seed per mission. This keeps the policy's size and form and destroys what it learned.

5.2 Units, streams and blocks

A *unit* is one policy evaluated on all 164 bugs under $S = 8$ tie-break streams. Its score is the mean number of bugs solved across the streams. Each Stage 3 seed defines a *block*. Within a block, r0, C6, C6f, C6r and the control share streams (common random numbers), so every contrast is paired. Each block has seven units:

- at budget 12: r0, C6, C6f, C6r and the control;
- at budget 48: r0 and C6.

That makes 168 units, $168 \times 8 \times 164 = 220{,}416$ repair attempts in all.

5.3 Primary endpoint and decision rule

For each seed s,

$$\Delta_s = \text{solved}(\text{C6 r32},\ s) - \text{solved}(\text{r0},\ s) \quad \text{at budget 12.}$$

TRANSFER is declared if and only if all three of these hold:

1. the one-sided exact Wilcoxon signed-rank p-value [16] is below 0.01 (zeros dropped, exact under ties; the same implementation as Stage 3);
2. the mean Δ is at least 1.0 bug;
3. the 95% percentile bootstrap interval of the mean [4] (B = 20,000, resampling seeds) lies above zero.

5.4 Secondary analyses

The following were registered and are reported whatever their outcome, without multiplicity correction. They support the interpretation; they are not claims.

Table 2: Registered secondary analyses.

	Contrast	Test
S1	C6f and C6r end states vs r0; pooled over 72 missions (descriptive)	one-sided
S2	C6 − C6f and C6 − C6r, mirroring Stage 3's causal tier	two-sided
S3	Primary contrast split by benchmark	one-sided
S4	Primary contrast at budget 48 (four times the training budget)	one-sided
S5	C6 trained vs parameter-permuted control	one-sided
S6	Synthetic gain vs real-bug gain across 72 missions	Spearman
S7	End states of a separate study (v6), if it reaches its confirmatory	conditional

5.5 What was known before the lock

All development used non-confirmatory data only, and all of it is disclosed:

- **Phase 0** (Section 4) counted reachability without any policy.
- **Phase 0b** evaluated r0 and 214 development policies on the 164 bugs with four streams each. The development policies came from Stage 3 Gate 1 missions, later studies' gate missions, development missions and campaign missions. r0 solved 21.5 bugs; the trained policies solved 25.3 on average (SD 2.7), and 94% of them beat r0.
- **Phase 0c** found that parameter-permuted controls of 23 development policies solved 19.2 bugs against 25.8 trained (22 of 23 pairs). At budget 48, r0 solved 34.5 and the Stage 3 Gate 1 C6 policies 37.0.

Simulated from the development deltas, the primary has power of about 1.00 at the development effect, 0.998 at half of it, and 0.37 at a quarter.

The lock recorded three predictions: the primary passes, S2 shows no difference, and S5 favours the trained policies. We state this plainly. The confirmatory test was not expected to surprise. Its value is that it measures transfer on the published end states under rules fixed in advance.

5.6 Timeline

Table 3: Timeline (UTC, 10 October 2026).

Event	Time
Phase 0 reachability; Phase 0b/0c development evaluation	before 07:50
Protocol locked; build frozen (hash `0b9617b8...`)	07:50
Gate 0	07:50–08:15
Confirmatory run (168 units)	08:18–08:40
Replay audit (fresh process, empty verdict store)	after 08:40
S7: evaluation of the later study's 72 end states (11 October)	04:40–05:10

6 Results I: Gate 0 and Integrity

Table 4: Gate 0. All six checks passed under the frozen build.

Check	Result	Criterion
G0-1 Benchmark integrity	3 fresh validations identical; 164 valid, equal to the frozen list	identical
G0-2 Port exactness	1,200 synthetic searches (300 tasks $\times$ 4 policies) and 2,121 diagnostic computations; **0 mismatches**	0
G0-3 Host equivalence	6,514 real-code sources (every buggy function and each of its one-step edits): **0 pool and 0 feature mismatches**	0
G0-4 Determinism	984 searches run twice with fresh executors and empty verdict stores; **0 differences**	identical
G0-5 Inputs	72 of 72 confirmatory state files match the published Stage 3 manifest	all match
G0-6 Headroom	r0 solves 20.6 of 164 (development streams)	5 to 100

Replay audit. The 15 budget-12 units of blocks 7101–7103 were rerun in a fresh process with an empty verdict store.

- All 15 units were identical, stream by stream.
- All 10,700 verdicts shared between the two stores agreed. That includes the 461 shared verdicts that took 0.25 s or longer, the only ones whose outcome could plausibly depend on timing.

1,146 of the confirmatory store's 25,794 verdicts were slow in that sense.

Other checks. The freeze verified after the run. There were no deviations from the protocol.

7 Results II: Transfer

Primary. Under the identical search and budget, the C6 end states solved **25.81** real bugs on average against **20.96** for r0: $\Delta = +4.85$, 95% CI [3.80, 5.84]. **23 of 24** seeds improved (1 declined) and the exact one-sided Wilcoxon gives $p = 3.0 \times 10^{-7}$ (Figure 2). All three preregistered conditions hold, so the verdict is **TRANSFER**.

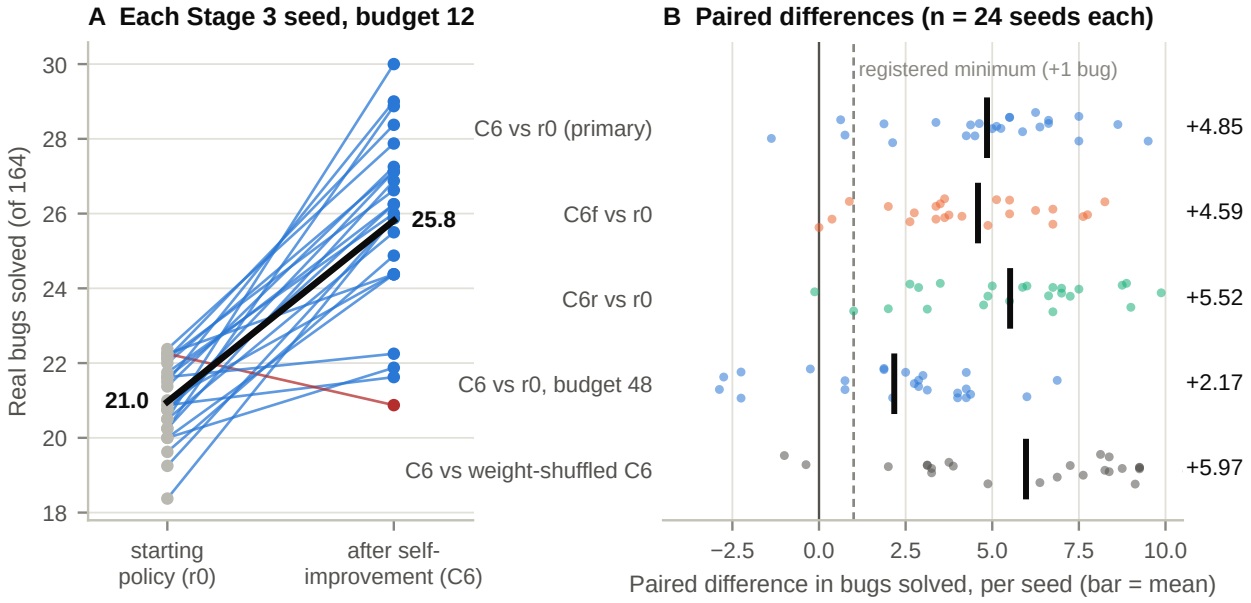


Figure 2: (A) Real bugs solved by each Stage 3 seed's starting policy and by its self-improved C6 end state, under the same search and budget. Thin lines are seeds; the thick line joins the means. (B) Per-seed paired differences for the primary contrast and four registered secondaries. Bars mark means, and the dashed line is the registered minimum effect.

Size of the effect. The gain on real bugs is 23% relative to r0. For the same 24 missions, the synthetic held-out solve rate rose 47% (0.274 to 0.401), so about half of the relative improvement carries over. Measured against the 55 bugs that any policy solved, r0 finds 38% and the self-improved policy 47%.

Table 5: All registered contrasts. Means are in bugs (of 164) with 95% bootstrap intervals; n = 24 seeds. Up/down/tied counts seeds.

	Contrast	Mean	95% CI	Up/down/tied	p
P1	C6 r32 − r0	**+4.85**	[3.80, 5.84]	23 / 1 / 0	3.0×10^{-7}
S1	C6f r32 − r0	+4.59	[3.57, 5.68]	23 / 0 / 1	1.2×10^{-7}
S1	C6r r32 − r0	+5.52	[4.47, 6.52]	23 / 1 / 0	1.2×10^{-7}
S2	C6 − C6f	+0.26	[−0.79, 1.33]	12 / 11 / 1	0.68 (two-sided)
S2	C6 − C6r	−0.67	[−1.96, 0.61]	10 / 14 / 0	0.35 (two-sided)
S3	HumanEvalFix only	+5.08	[4.04, 6.07]	23 / 1 / 0	1.2×10^{-7}
S3	QuixBugs only	−0.23	[−0.52, 0.08]	5 / 18 / 1	0.97
S4	Budget 48: C6 − r0	+2.17	[1.10, 3.18]	19 / 5 / 0	4.3×10^{-4}
S5	C6 − permuted control	+5.97	[4.67, 7.21]	22 / 2 / 0	3.0×10^{-7}

All three conditions transfer, and they do not differ. Each condition's end states beat r0 by 4.6 to 5.5 bugs. Across all 72 missions, 69 are positive. The two arm contrasts are small and not significant, in the same direction as Stage 3's causal tier: C6 is no better than its frozen- mechanism and random-adoption controls on real bugs either.

The gain survives a larger budget. At four times the training budget, r0 solves 34.37 bugs and C6 36.54 ($+2.17$, $p = 4.3 \times 10^{-4}$). The advantage shrinks, as expected when the search can afford to try more of the candidates the policy deprioritises, but it does not disappear.

The gain comes from what was learned. Permuting the learned parameters drops the policy to 19.84 bugs, below r0. The trained policy beats its own permutation by 5.97 bugs in 22 of 24 seeds. Simply having *some* fixed, non-random ordering is not enough; it is the learned content that transfers.

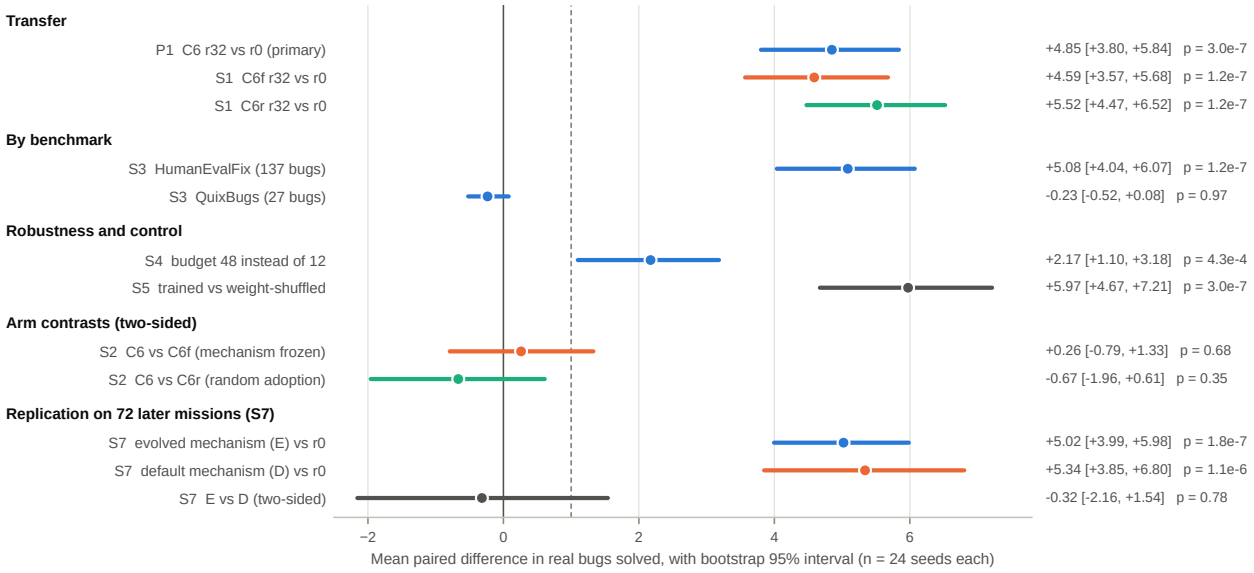


Figure 3: Registered contrasts: means with bootstrap 95% intervals. The dashed line is the registered minimum effect for the primary.

Replication on independent missions (S7). The protocol registered a conditional secondary on the end states of a later preregistered study of the same system (v6), to be evaluated with the identical frozen pipeline if that study reached its confirmatory. It did: 72 further 32-round missions on 24 fresh seeds. They used a later generation of the mission code and three conditions for the improvement mechanism: evolved by a validated campaign (E), default (D), and randomly drifted (R). On synthetic tasks that study's evolved mechanism did slightly *worse* than the default (-1.12 pp). On real bugs (Table 6), every condition's self-improved policies again beat their starting policy by about five bugs (68 of 72 missions improved), and the evolved and default mechanisms again did not differ. Both findings of this paper, transfer of the loop's product and no added benefit from rewriting the mechanism, thus replicate on an independent set of missions.

Table 6: S7: the later study's 72 confirmatory end states on the same 164 real bugs (8 streams, budget 12, own r0 per seed). Means in bugs; n = 24 seeds.

Contrast	Mean	95% CI	Up/down/tied	p
D r32 − r0	+5.34	[3.85, 6.80]	22 / 2 / 0	1.1×10^{-6}
E r32 − r0	+5.02	[3.99, 5.98]	23 / 1 / 0	1.8×10^{-7}
R r32 − r0	+4.81	[3.76, 5.91]	23 / 1 / 0	1.8×10^{-7}
E − D	−0.32	[−2.16, 1.54]	12 / 11 / 1	0.78 (two-sided)
E − R	+0.21	[−1.29, 1.71]	13 / 11 / 0	0.73 (two-sided)

8 Results III: Where the Gain Is

By benchmark. The transfer is entirely on HumanEvalFix (+5.08 bugs); QuixBugs shows none (−0.23, 5 seeds up and 18 down). Only 4 of the 27 QuixBugs bugs are one-step reachable. The programs are classic algorithms, and most of their faults lie outside the one-step edit set. On the few that are reachable, the self-improved policies rank the fix slightly lower than a random order does.

By bug type. Within HumanEvalFix the gain is in value misuse (6.48 to 9.76 expected solves) and operator misuse (6.65 to 8.00). These two categories hold 34 of the 51 reachable HumanEvalFix bugs, and they are the kinds of fault the synthetic domain injects: wrong constants, wrong operators, wrong comparisons. Excess logic declines slightly (3.32 to 2.87).

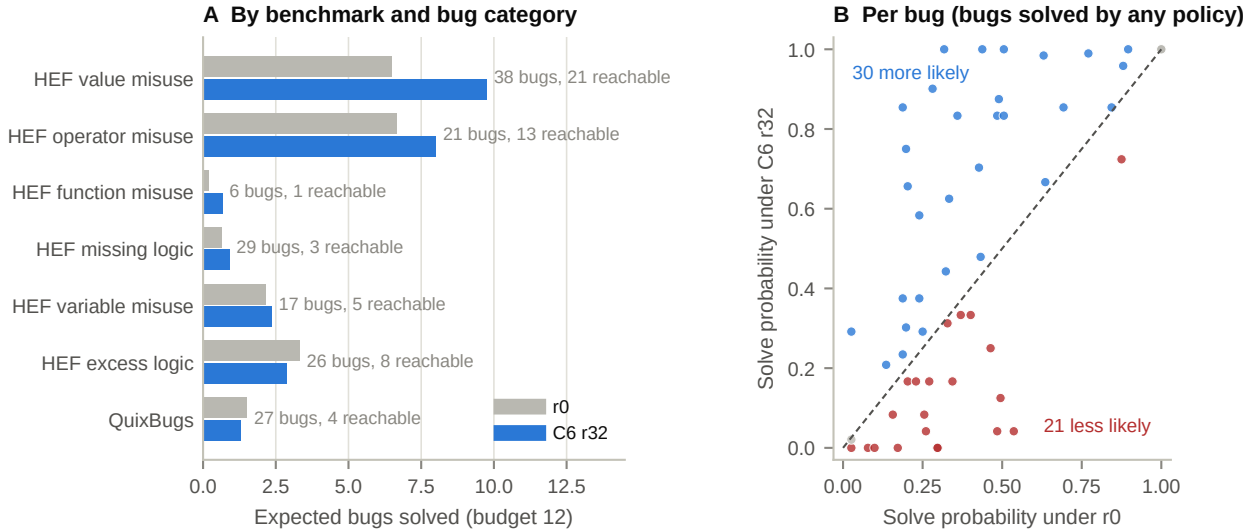


Figure 4: (A) Expected bugs solved under r0 and the C6 end states, averaged over 24 seeds and 8 streams, by benchmark and HumanEvalFix (HEF) bug category. "Reachable" means a one-step fix exists in the frozen edit set. (B) Per-bug solve probability, r0 against C6, for every bug either policy ever solved.

By bug. The learned ordering trades some bugs for others. 30 bugs became more likely to be solved and 21 less likely (Figure 4B), and the gains are larger than the losses.

Ceiling. 55 distinct bugs were solved by at least one unit, and 50 of them are one-step reachable. The other five were solved only through multi-step climbs through partial improvements. Five one-step-reachable bugs were never solved. Each of these five is among the 15 whose faulty program already passes every visible example, so the search had nothing to climb.

Synthetic gain does not predict real gain. Across the 72 missions, the size of a mission's synthetic improvement is unrelated to its real-bug improvement (Spearman $\rho = -0.01$; Figure 5). Transfer is a shared property of the trained policies, not a scaled-down copy of each mission's own synthetic gain.

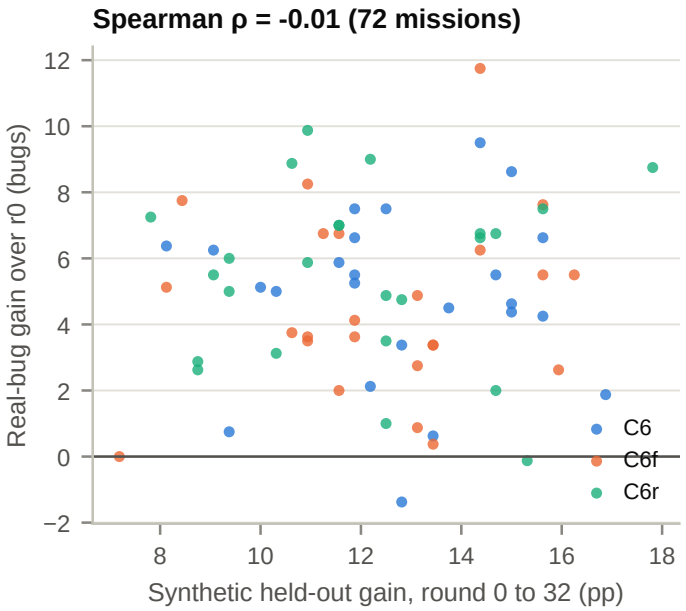


Figure 5: Each mission's synthetic held-out gain (round 0 to 32) against its real-bug gain over r0.

9 Discussion

What the result shows. The policies FELIX-C3 produced by bounded self-improvement on generated tasks repair measurably more real bugs from two external benchmarks than the system's own starting point: about five more bugs out of 164, or 23%, under the identical search. The effect holds across seeds, across all three conditions and at a larger budget, it vanishes when the learned content is scrambled, and it replicates on 72 independent missions from a later study. The self-improvement reported in Stage 3 is therefore not only fit to the synthetic task generator. Part of it is repair competence that carries over to faults written by other people for other purposes.

What it does not show. Three limits follow directly from the registered secondaries.

First, what transfers is the product of the improvement loop, not of *recursive* self-improvement. The three conditions differ only in what happens to the improvement mechanism, and they transfer equally. Stage 3 found the same on synthetic tasks, and so does the replication (S7). The Level-4 classification rests on the system rewriting its own mechanism, but the benefit that reaches real bugs is one the system obtains whether or not that rewriting is evidence-driven, or happens at all.

Second, the transfer is narrow. It sits in the fault types closest to the synthetic domain, and it is absent on algorithmic code with loop and index faults.

Third, the system is small. With a one-step edit set and a budget of 12, it reaches at most 55 of the 164 bugs and solves about 26. It is not a competitive program-repair system, and these numbers should not be compared with published repair tools or with repair results from language models.

Relation to the Level-4 claim. Liu et al.'s Level 4 requires sustained progress *within a bounded domain* [10]. Nothing here changes the classification of FELIX-C3. What changes is how that progress should be read. Stage 3 could show only that the system got better at its own domain. This study shows that roughly half of that relative gain is real repair skill on independent code. External transfer is not a criterion of the definition, but it is the property that makes a self-improvement claim worth having. We suggest that evaluations of self-improving systems report it alongside in-domain progress.

Why this design. The test uses only artefacts that existed and were hash-committed before the question was asked: the published end states, their manifest, and the frozen search. The single new component, the executor, was shown to be exact before use. This leaves little room for the result to have been engineered. There were no benchmark-specific features, no tuning of the edit set, no selection of policies, and no choice of budget after seeing data. What room remains is disclosed: the development evaluation on non-confirmatory policies predicted the result, and we said so at the lock.

10 Limitations

- **Ceiling and scale.** The one-step edit set caps the system near 55 of 164 bugs, and the absolute effect is about five bugs. Transfer to larger programs, multi-line faults or other languages is untested.
- **Benchmark coverage.** 31 tasks were excluded: 27 HumanEvalFix tasks whose assertions are not literal I/O pairs, 3 QuixBugs tasks whose reference fails its own JSON tests, and 1 with too few cases. QuixBugs contributes only 27 bugs, with 4 reachable.
- **Split.** The fixed first-half/second-half split makes 15 bugs invisible to the search. A different split would change the absolute counts. It applies identically to every policy, so it cannot bias the paired contrasts.
- **Timeouts.** Verdicts that depend on a time limit could in principle vary with machine load. The confirmatory ran alongside another study. The replay agreed on every shared slow verdict, but the replay covered 15 of 168 units.
- **Registration.** The protocol was locked by hash and posted to the author's project records before the confirmatory, not on a public registry with an independent timestamp. The audit package lets a reader verify every hash and rerun every unit, but the order of events rests on those records.
- **Prediction.** Development data predicted a pass. The confirmatory is a check on the published states, not an independent discovery.
- **One system.** This is one self-improving system on one external task family. It says nothing about self-improvement in general, or about systems that use language models.

11 Reproducibility, Integrity and Use of AI Assistance

Artefacts. The frozen build (hash `0b9617b8...`) contains the benchmark loader, the executor, the search port, the runner and the unchanged task-domain, policy and statistics modules. The run directory holds the Gate 0 record, all 168 checksummed units, the verdict store with timings, the results file and the replay record. Every hash is listed in Appendix B. A confidential audit package with the code and runs is available to reviewers on request, as for Stage 3. The commands `verify`, `gate0`, `confirm`, `evaluate`, `replay` and `confirm-v6` regenerate every number in this paper.

Versions. Version 1.0 reported the primary analysis and secondaries S1–S6. Version 1.1 adds S7, which was registered at the lock as conditional and could only run once the later study reached its confirmatory. Nothing else changed.

Pipeline dry run. Before the freeze, the whole pipeline (Gate 0, confirmatory, evaluation, replay) was run end to end on three Stage 3 Gate 1 seeds, under a separate dry-run freeze. Those numbers are not part of the study.

Author contributions and use of AI assistance. The research direction belongs to the author. So do the decisions to test external transfer, to register the test, to proceed after Gate 0, to publish, and to disclose AI assistance. The study design, protocol, implementation (the executor, the port and the runner), execution and monitoring, analysis, figures and drafting of this paper were carried out with substantial assistance from an AI system (Claude, Anthropic), working under the author's direction. The AI is not part of the system under test. No language model takes part in the repair loop, in producing the policies, or in deciding whether a candidate repair is accepted. Every decision in the loop is made by the frozen search and the policies, and every verdict by executing the benchmark's own tests.

12 Conclusion

Self-improvement claims are easy to make inside the domain where the system improves, and hard to test outside it. We tested one. The repair policies that a Level-4 bounded self-improving system produced on generated tasks, run unchanged on 164 real bugs from QuixBugs and HumanEvalFix, solved 23% more than the system's own starting policy ($p = 3.0 \times 10^{-7}$). The result holds across seeds and conditions, at a larger budget, and only when the learned content is intact, and it replicates on 72 independent missions.

The same test draws three boundaries:

- the transfer is narrow: there is none on QuixBugs;
- it is small in absolute terms: the system can reach at most 55 of the 164 bugs;
- it is not attributable to the system rewriting its own improvement mechanism: all three conditions transfer equally.

The progress is real, and some of it leaves the sandbox. Whether the recursion itself pays is a separate question, and still an open one.

References

[1] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. arXiv:2107.03374, 2021.

[2] Vidroha Debroy and W. Eric Wong. Using mutation to automatically suggest fixes for faulty programs. In *Third International Conference on Software Testing, Verification and Validation (ICST)*, pages 65–74, 2010. doi: 10.1109/ICST.2010.66.

[3] Richard A. DeMillo, Richard J. Lipton, and Frederick G. Sayward. Hints on test data selection: Help for the practicing programmer. *Computer*, 11(4):34–41, 1978. doi: 10.1109/C-M.1978.218136.

[4] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.

[5] Junai P. Felix. Level-4 bounded recursive self-improvement under a published definition: A preregistered test of a self-referential improvement mechanism. Zenodo, record 23138422, 2026.

[6] Yue Jia and Mark Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678, 2011. doi: 10.1109/TSE.2010.62.

[7] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012. doi: 10.1109/TSE.2011.104.

[8] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[9] Derrick Lin, James Koppel, Angela Chen, and Armando Solar-Lezama. QuixBugs: A multi-lingual program repair benchmark set based on the Quixey challenge. In *Proceedings Companion of the 2017 ACM SIGPLAN International Conference on Systems, Programming, Languages, and Applications: Software for Humanity (SPLASH Companion)*, pages 55–56, 2017. doi: 10.1145/3135932.3135941.

[10] Shuaiqi Liu, Zhengkai Lin, Yuxiang Zhang, Yuanyi Ren, Yue Wu, Yongbin Li, Zheng Wang, Zhihang Fu, and Jieping Ye. The path to recursive self-improving agents: Foundation, framework, and future directions. Preprints.org, preprint 202608.0051, version 1, 2026. Not peer reviewed.

[11] Fan Long and Martin Rinard. Automatic patch generation by learning correct code. In *Proceedings of the 43rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 298–312, 2016. doi: 10.1145/2837614.2837617.

[12] Martin Monperrus. Automatic software repair: A bibliography. *ACM Computing Surveys*, 51(1): 17:1–17:24, 2018. doi: 10.1145/3105906.

[13] Niklas Muennighoff, Qian Liu, Armel Zebaze, Qinkai Zheng, Binyuan Hui, Terry Yue Zhuo, Swayam Singh, Xiangru Tang, Leandro von Werra, and Shayne Longpre. OctoPack: Instruction tuning code large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.07124.

[14] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. The preregistration revolution. *Proceedings of the National Academy of Sciences*, 115(11):2600–2606, 2018. doi: 10.1073/pnas.1708274114.

[15] Joelle Pineau, Philippe Vincent-Lamarre, Koustuv Sinha, Vincent Larivière, Alina Beygelzimer, Florence d'Alché Buc, Emily Fox, and Hugo Larochelle. Improving reproducibility in machine learning research (A report from the NeurIPS 2019 reproducibility program). *Journal of Machine Learning Research*, 22(164):1–20, 2021.

[16] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.

A Per-Seed Results

Table 7: Bugs solved (of 164) per Stage 3 seed, mean over 8 tie-break streams. Budget 12 unless marked b48.

Seed	r0	C6	C6f	C6r	Permuted	r0 b48	C6 b48	Δ (P1)
7101	21.75	26.00	22.12	28.38	17.62	34.62	37.75	+4.25
7102	22.00	27.12	22.00	28.00	23.88	34.88	37.88	+5.12
7103	21.38	27.25	29.62	31.25	19.00	34.75	37.25	+5.88
7104	20.25	28.88	25.75	29.00	20.12	35.75	37.88	+8.62
7105	20.00	21.88	23.38	26.75	22.25	33.50	36.38	+1.88
7106	18.38	25.88	30.12	27.25	22.00	32.00	36.25	+7.50
7107	22.25	24.38	24.25	29.25	15.12	34.50	31.75	+2.12
7108	22.38	27.88	27.88	24.38	24.75	34.88	39.12	+5.50
7109	20.88	21.62	25.00	26.75	22.62	35.25	33.00	+0.75
7110	22.12	26.62	25.62	24.75	17.38	37.62	37.38	+4.50
7111	22.12	28.38	29.88	27.12	17.75	35.12	37.88	+6.25
7112	20.50	25.50	25.62	27.75	21.75	34.38	36.25	+5.00
7113	20.75	26.25	21.62	20.62	19.38	33.88	34.62	+5.50
7114	19.62	24.88	24.50	28.62	17.25	32.50	38.50	+5.25
7115	19.25	25.88	26.00	26.25	17.75	34.62	35.38	+6.62
7116	22.25	20.88	24.88	27.12	17.62	35.00	32.75	-1.38
7117	21.62	26.25	25.25	26.38	17.00	34.25	38.25	+4.62
7118	21.50	29.00	25.25	28.25	19.88	35.12	38.00	+7.50
7119	20.75	27.12	27.50	23.62	18.75	32.62	39.50	+6.38
7120	20.25	26.88	23.62	23.75	24.88	33.00	37.38	+6.62
7121	21.00	24.38	23.75	22.00	19.50	34.38	38.38	+3.38
7122	20.50	30.00	28.12	28.00	22.75	34.75	39.00	+9.50
7123	21.62	22.25	25.25	27.12	19.12	35.00	32.12	+0.62
7124	20.00	24.38	26.25	23.12	18.00	32.50	34.38	+4.38
Mean	20.96	25.81	25.55	26.48	19.84	34.37	36.54	+4.85

B Hashes

Table 8: SHA-256 hashes (QuixBugs: git commit) of the frozen build, inputs and outputs.

Item	Hash
Freeze hash	0b9617b8d2fd774dda7a6fec001fbb9d429888617ceb3730a4874a7fea37aec0
bench.py	21c0dd50d5d57e1113eefa70f208597408bdee48db4dd7eb32ba64984b349a2a
executor.py	af2c4582a1cbac0261e1e2f4f2406eb46d88808e3641ec1a1ba05525bd75f3ba
host.py	829e8fa50080739bd22bea0ab403c2640294350c5950493e8428043b52c4d180
levels.py	ef4107850d02f11827a330a0e5cff05bd5c500423821a329e07c0875d048e34b
realhost.py	66debeb74bacaa4a4c33a301b9518ad6a211333bb76c5f266fae8e68c0822b48
runner7.py	53b49f47ef235f561a17a4dd9b624e645c6bccf16208cbed7b0f62b7d81123d9
stats.py	ee8c288d9207f1ded2a896826c1a5da33e7f0c8c4033681f4753e90a3cb7d0c6
Stage 3 host.py	9ce16966d7b973ada54063b3a0631da2878b3e297a8775739d8ff3823d79ef49
Stage 3 audit manifest	ac7e7829952da25ae8de5bf9f7c9b1ee5978c001116b37494db5aa751db30dca
Valid-task content	38f045a12ec0c5e34b7d39841cbd8f0ed2db4bcb4207d1ded483c56a4fc8e443
HumanEvalFix parquet	ed5f15d789156e21222bfcd556c425a39042355c84ae1e8b058abd6a3d7f8075
QuixBugs git commit	4257f44b0ff1181dedaedee6a447e133219fcebf
FREEZE.json	c078eac546958777b4a85a7a91aadea5a22bbe63fab4be508c93769cb1416e22
runs/gate0.json	767a921f66d8b38b848db787ff91d9aa5962d0309c25d38a6ab253d2aee2e0e9
runs/results.json	f1c95ca742edbb63bfc54e7f9fbf5452b497aefa04caa399f4dfe36cbcf9c289
runs/replay.json	5c2da64ac60fc9b2ed05c3bf396c9a1048c0607edeb774f028e34c3646f02221
S7 results (results_v6.json)	313be684c7881b817cb98c958077d00fc52deda4ff0a153fcb89436450c327e7
Verdict store	2b7191f564e717a21c5bf2816bc31f34d62bfea0f53e8e5f04bcc7f1b3114e8e